

```
In [ ]: import marimo as mo
```

```
In [ ]: import numpy as np
import pandas as pd
import altair as alt
alt.renderers.enable("svg")
```

```
RendererRegistry.enable('svg')
```

Efficient Transformer Attention Mechanisms: Methods and Trade-offs

Executive Summary

The standard (full) self-attention mechanism has **quadratic** time and memory complexity:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \in \mathcal{O}(n^2 \cdot d)$$

For a sequence of length n and head dimension d_k , the QK^T product alone produces an $n \times n$ matrix. At $n = 16,384$ this is already **268 M** entries — far beyond what fits in GPU memory for reasonable batch sizes.

Efficient attention research asks: *can we approximate or restructure this computation so that complexity drops to $\mathcal{O}(n \log n)$ or $\mathcal{O}(n)$ while preserving most of the modelling power?*

This notebook surveys the leading families of solutions, compares their theoretical complexities, visualises their sparsity patterns, and benchmarks them on the **Long Range Arena (LRA)** suite.

Categories of Efficient Attention

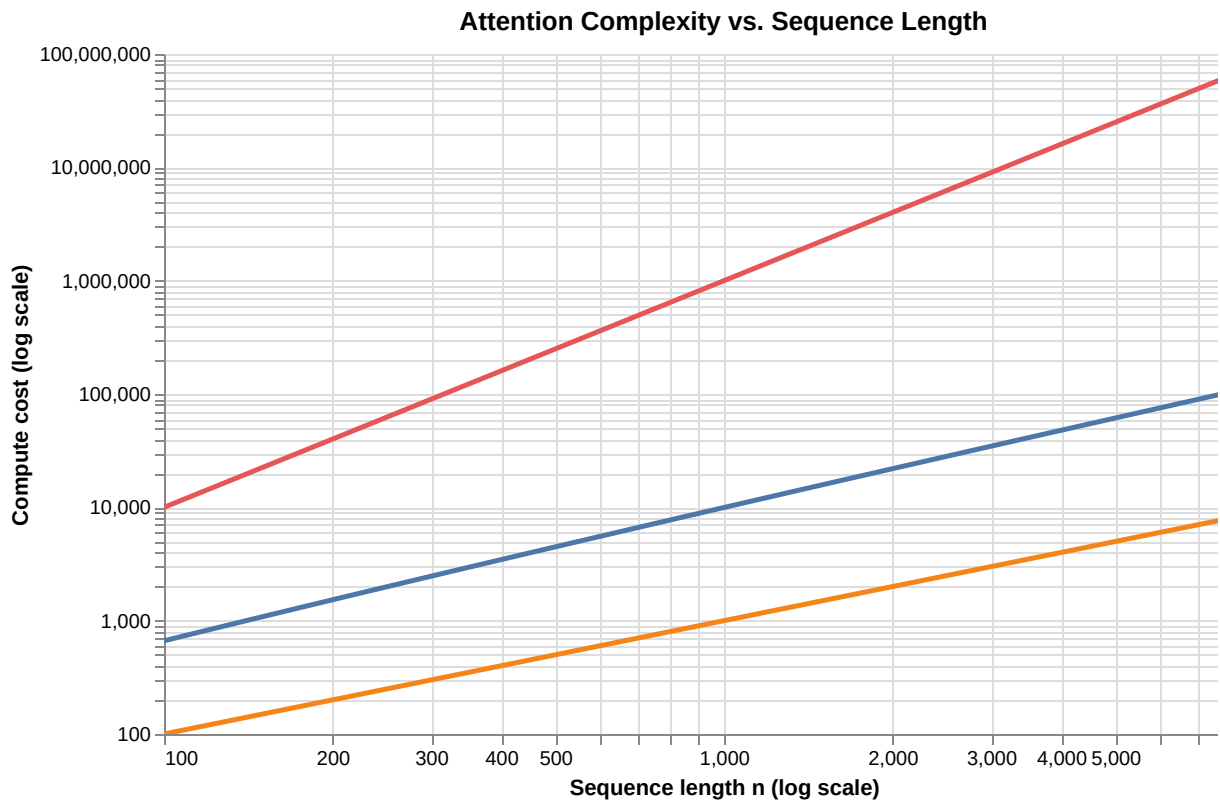
Family	Key Idea	Representative Methods
Sparse / Structural	Attend only to a fixed sparse subset of tokens	Longformer, BigBird, Sparse Transformer
Low-Rank / Linear	Project keys/values to a smaller dimension	Linformer
Kernel / Random Features	Replace softmax with a kernel that factorises	Performer, Random Feature Attention

Family	Key Idea	Representative Methods
Hashing / Clustering	Group similar queries and keys before attending	Reformer (LSH), Routing Transformer
Nyström Approximation	Use landmark points to reconstruct the full matrix	Nyströmformer
Recurrent / State-Space	Reformulate as an RNN or SSM for $\mathcal{O}(1)$ steps	Linear Transformer, S4, Mamba

Each family makes different **accuracy vs. speed** trade-offs and is suited to different sequence lengths and downstream tasks.

```
In [ ]: _n_vals = np.logspace(2, 4, 200)
        _rows = []
        for _n in _n_vals:
            _rows.append({"n": _n, "cost": _n ** 2, "method": "O(n²) – Full"})
            _rows.append({"n": _n, "cost": _n * np.log2(_n), "method": "O(n log n) –"})
            _rows.append({"n": _n, "cost": _n, "method": "O(n) – Linear/Performer"})
        complexity_df = pd.DataFrame(_rows)

        complexity_chart = (
            alt.Chart(complexity_df)
            .mark_line(strokeWidth=2.5)
            .encode(
                x=alt.X("n:Q", scale=alt.Scale(type="log"), title="Sequence length r",
                y=alt.Y("cost:Q", scale=alt.Scale(type="log"), title="Compute cost (",
                color=alt.Color("method:N", title="Complexity class",
                                scale=alt.Scale(scheme="tableau10")),
                tooltip=["method:N", alt.Tooltip("n:Q", format=".0f"),
                        alt.Tooltip("cost:Q", format=".2e")],
            )
            .properties(
                title="Attention Complexity vs. Sequence Length",
                width=560,
                height=340,
            )
            .interactive()
        )
        complexity_chart
```



```
In [ ]: _N = 32

def _full_pattern(n: int) -> np.ndarray:
    return np.ones((n, n))

def _longformer_pattern(n: int, window: int = 4) -> np.ndarray:
    mat = np.zeros((n, n))
    for i in range(n):
        for j in range(max(0, i - window), min(n, i + window + 1)):
            mat[i, j] = 1.0
    return mat

def _reformer_pattern(n: int, n_blocks: int = 4) -> np.ndarray:
    mat = np.zeros((n, n))
    block = n // n_blocks
    for b in range(n_blocks):
        s, e = b * block, (b + 1) * block
        mat[s:e, s:e] = 1.0
    return mat

def _bigbird_pattern(n: int, window: int = 3, n_global: int = 2, n_random: int = 1) -> np.ndarray:
    rng = np.random.default_rng(42)
    mat = _longformer_pattern(n, window)
    for i in range(n):
        rand_cols = rng.choice(n, size=n_random, replace=False)
        mat[i, rand_cols] = 1.0
    mat[:n_global, :] = 1.0
    mat[:, :n_global] = 1.0
    mat[-n_global:, :] = 1.0
    mat[:, -n_global:] = 1.0
    return mat
```

```

def _mat_to_df(mat: np.ndarray) -> pd.DataFrame:
    rows = []
    n = mat.shape[0]
    for i in range(n):
        for j in range(n):
            rows.append({"query": i, "key": j, "attention": float(mat[i, j])})
    return pd.DataFrame(rows)

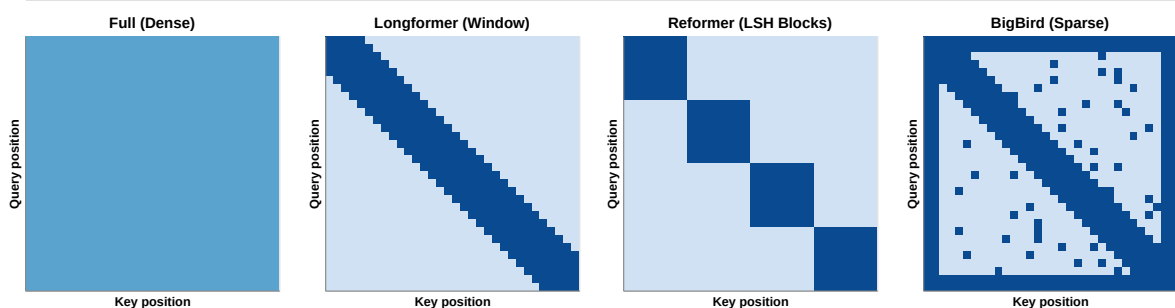
def _heatmap(df: pd.DataFrame, title: str) -> alt.Chart:
    return (
        alt.Chart(df)
        .mark_rect()
        .encode(
            x=alt.X("key:0", axis=alt.Axis(labels=False, ticks=False), title="key:0",
            y=alt.Y("query:0", axis=alt.Axis(labels=False, ticks=False), title="query:0",
            color=alt.Color("attention:Q",
                scale=alt.Scale(scheme="blues"),
                legend=None),
            tooltip=["query:0", "key:0", alt.Tooltip("attention:Q", format="%0.2f")]
        )
        .properties(title=title, width=220, height=220)
    )

_patterns = {
    "Full (Dense)": _full_pattern(_N),
    "Longformer (Window)": _longformer_pattern(_N),
    "Reformer (LSH Blocks)": _reformer_pattern(_N),
    "BigBird (Sparse)": _bigbird_pattern(_N),
}

_charts = [_heatmap(_mat_to_df(mat), name) for name, mat in _patterns.items()]

attention_heatmaps = mo.hstack([
    mo.hstack(_charts[:2]),
    mo.hstack(_charts[2:]),
], justify="center")
attention_heatmaps

```



Linformer — Low-Rank Projection

Concept

Linformer (Wang et al., 2020) observes that the attention matrix is approximately **low-rank** in practice. It projects the $n \times d$ key and value matrices

down to $k \times d$ using learned linear projections $E, F \in \mathbb{R}^{k \times n}$ (also written W_K, W_V in some formulations):

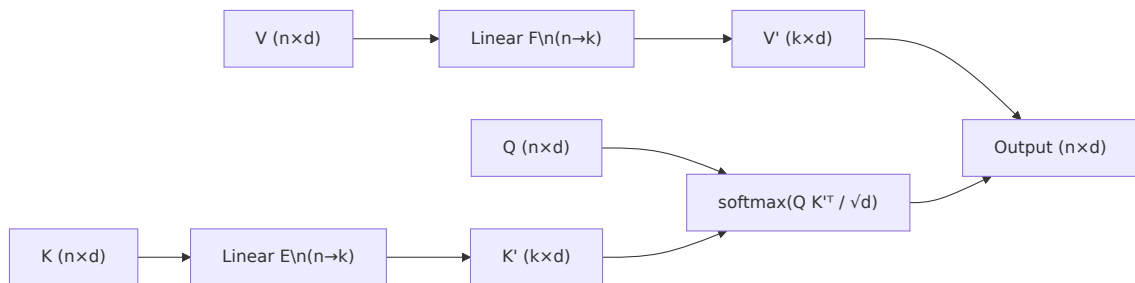
$$K' = EK, \quad V' = FV \quad (k \ll n)$$

The approximated attention then becomes:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK'^T}{\sqrt{d_k}}\right) V'$$

- **Complexity:** $\mathcal{O}(nk)$ — effectively $\mathcal{O}(n)$ for fixed k
- **Memory:** $\mathcal{O}(nk)$ vs. $\mathcal{O}(n^2)$

Data-flow



Empirical performance

On LRA, Linformer reaches **~53.9** average accuracy — better than the full transformer baseline, likely because the implicit regularisation from the projection helps on some tasks.

Pros: Simple to implement; large memory savings; drop-in replacement for standard attention. **Cons:** Fixed projection size k must be tuned; performance degrades on tasks requiring precise positional information (e.g., retrieval); projection matrices are not input-dependent. **Best for:** Long documents where approximate attention suffices (classification, summarisation).

Performer / Random Feature Attention (RFA)

Concept

Performers (Choromanski et al., 2021) replace the softmax kernel with a **random feature map** $\varphi : \mathbb{R}^d \rightarrow \mathbb{R}^r$ such that:

$$\exp\left(\frac{Q_i^T K_j}{\sqrt{d}}\right) \approx \varphi(Q_i)^T \varphi(K_j)$$

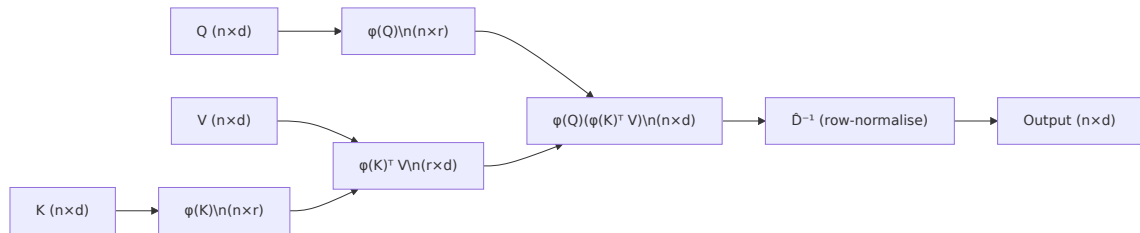
This factorisation lets the computation be reordered:

$$\text{Attn}(Q, K, V) = \hat{D}^{-1} [\varphi(Q) (\varphi(K)^T V)]$$

where $\hat{D} = \text{diag}(\varphi(Q) \varphi(K)^T \mathbf{1}_n)$.

- **Complexity:** $\mathcal{O}(nr \cdot d)$ — linear in n for fixed r
- **FAVOR+** uses orthogonal random features for lower variance

Data-flow



Pros: Unbiased (or positive-biased) approximation; easy causal masking; theoretically grounded. **Cons:** Approximation variance can be high; accuracy on retrieval tasks is notably lower than full attention; requires tuning r . **Best for:** Very long sequences where exact attention is infeasible; streaming / autoregressive settings.

Reformer — Locality-Sensitive Hashing (LSH) Attention

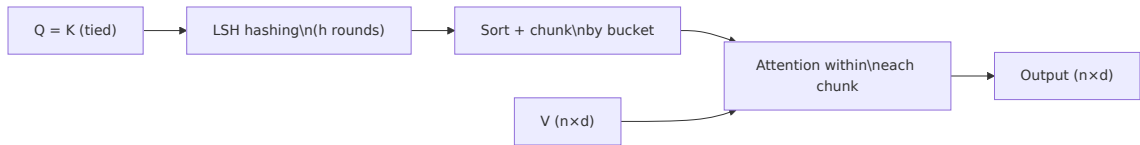
Concept

Reformer (Kitaev et al., 2020) reduces the attention cost by attending only to tokens that are **likely to have high dot-product similarity** to each query. It uses **Locality-Sensitive Hashing** to bucket queries and keys into the same hash bucket, then performs attention within each bucket.

- Tokens are hashed with h rounds of random rotations so similar vectors collide
- Sequences are sorted by hash bucket and chunked
- Complexity: $\mathcal{O}(n \log n)$ (due to the sort)

Additionally Reformer uses **reversible residual layers** to cut memory from $\mathcal{O}(L \cdot n)$ to $\mathcal{O}(n)$ across L layers.

Data-flow



Pros: Sub-quadratic for long sequences; reversible layers save memory across depth. **Cons:** Requires $Q = K$ (tied); hash collisions can cause misses; slower wall-clock time than simpler linear methods; complex to implement correctly. **Best for:** Very long sequences (>8 K tokens) where the bucket structure matches the data.

Longformer — Sliding Window + Global Tokens

Concept

Longformer (Beltagy et al., 2020) uses a **sliding window** of size w so each token attends to its $w/2$ left and $w/2$ right neighbours, plus a small set of **global tokens** (e.g., [CLS]) that attend to and from all positions.

$$\text{Complexity} = \mathcal{O}(n \cdot w + n_g \cdot n) \approx \mathcal{O}(n \cdot w) \quad \text{for } n_g \ll n$$

Dilated windows can be used in deeper layers to increase the receptive field without increasing cost.

Data-flow



Pros: Simple and efficient; proven on document NLU (LED, LongFormer-base); local pattern + global context is a natural inductive bias. **Cons:** Fixed window limits long-range dependencies for non-global tokens; global token selection requires task-specific engineering.

BigBird — Sparse Attention with Theoretical Guarantees

Concept

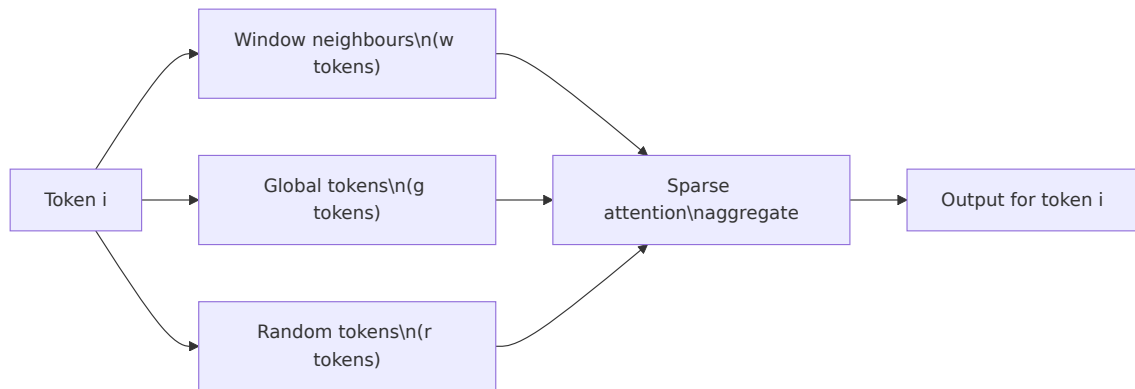
BigBird (Zaheer et al., 2020) combines **three** attention components:

1. **Local (window)** attention — each token attends to w neighbours
2. **Global** tokens — g special tokens attend to and from all positions
3. **Random** attention — each token attends to r uniformly sampled positions

This sparse pattern is provably a universal approximator of full attention and is a **complete graph** in the graph-theoretic sense.

$$\text{Complexity} = \mathcal{O}(n(w + g + r)) = \mathcal{O}(n) \quad \text{for fixed } w, g, r$$

Data-flow



Pros: Strong LRA results (best among sparse methods); theoretical guarantees; flexible — can approximate any full-attention pattern. **Cons:** More complex to implement than pure window attention; random attention introduces non-determinism; hyperparameters w, g, r need tuning.

Nyströmformer — Nyström Matrix Approximation

Concept

Nyströmformer (Xiong et al., 2021) applies the **Nyström method** from numerical linear algebra to approximate the $n \times n$ attention matrix using $m \ll n$ landmark points:

$$A \approx A_{nm} A_{mm}^+ A_{mn}$$

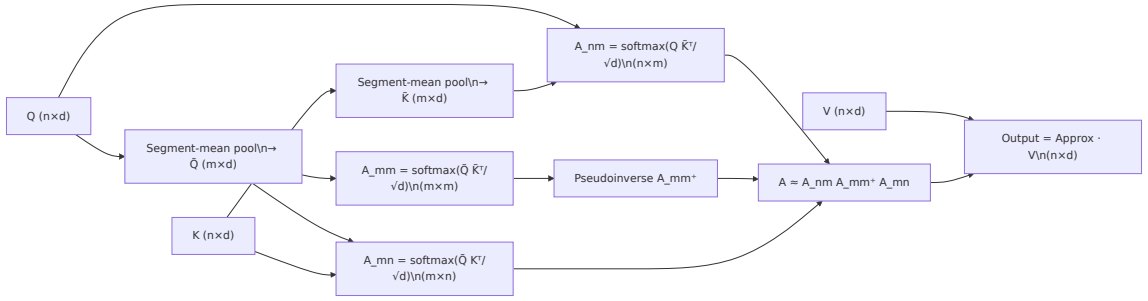
where A_{mm} is the $m \times m$ softmax sub-matrix between landmark queries and keys, and $(\cdot)^+$ is the Moore-Penrose pseudoinverse.

Landmark points are selected by **segment-mean pooling** of queries and keys.

- **Complexity:** $\mathcal{O}(nm)$ — linear for fixed m

- Pseudoinverse computed once per forward pass: $\mathcal{O}(m^3)$ (negligible for small m)

Data-flow



Pros: Strong accuracy (competitive with BigBird on LRA); theoretically principled; no special hardware needed. **Cons:** Pseudoinverse computation adds overhead; segment pooling discards fine-grained positional information.

Linear Transformer — Kernel Trick

Concept

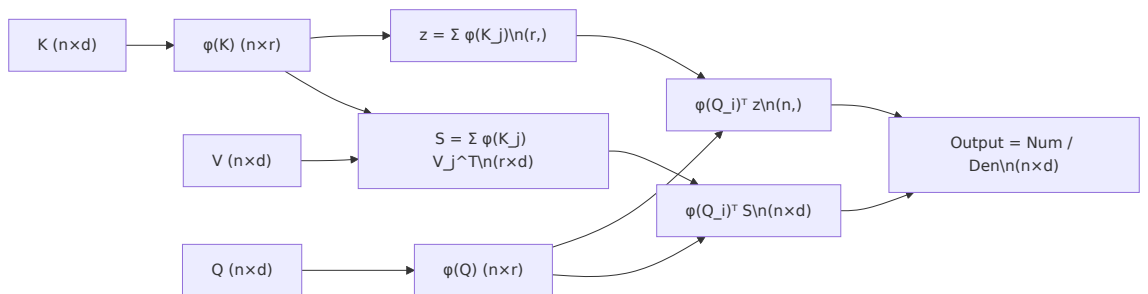
Katharopoulos et al. (2020) show that if the softmax is replaced by a **feature-map kernel** $\kappa(Q_i, K_j) = \varphi(Q_i)^T \varphi(K_j)$, the associativity of matrix products allows reordering:

$$\text{Attn}(Q, K, V)_i = \frac{\sum_j \varphi(Q_i)^T \varphi(K_j) V_j}{\sum_j \varphi(Q_i)^T \varphi(K_j)} = \frac{\varphi(Q_i)^T \left(\sum_j \varphi(K_j) V_j^T \right)}{\varphi(Q_i)^T \left(\sum_j \varphi(K_j) \right)}$$

The inner sums $S = \sum_j \varphi(K_j) V_j^T \in \mathbb{R}^{r \times d}$ and $z = \sum_j \varphi(K_j) \in \mathbb{R}^r$ can be **computed once and reused**, giving $\mathcal{O}(nr)$ total cost.

A common choice is $\varphi(x) = \text{elu}(x) + 1$ (element-wise).

Data-flow



Pros: Exact $\mathcal{O}(n)$ — no approximation hyperparameters; trivial causal masking via prefix sums (streaming RNN formulation). **Cons:** The feature map is a coarse approximation of softmax; lower accuracy on tasks needing sharp attention (e.g., retrieval); `elu+1` can cause numerical instability. **Best for:** Autoregressive generation at very long horizons; on-device / edge inference.

```
In [ ]: _data = {
    "Method": [
        "Full Transformer", "Linformer", "Performer", "Reformer",
        "Longformer", "BigBird", "Nyströmformer", "Linear Transformer",
    ],
    "Complexity": [
        "O(n2)", "O(n)", "O(n)", "O(n log n)",
        "O(n·w)", "O(n)", "O(n)", "O(n)",
    ],
    "Memory": [
        "O(n2)", "O(nk)", "O(nr)", "O(n log n)",
        "O(n·w)", "O(n)", "O(nm)", "O(r·d)",
    ],
    "Approximation": [
        "None (exact)", "Low-rank projection", "Random features",
        "LSH bucketing", "Local + global", "Local + random + global",
        "Nyström landmarks", "Kernel feature map",
    ],
    "Best For": [
        "Short sequences, fine-tuning", "Long docs (classification)", "Strea
        "Very long sequences", "Document NLU", "Long docs (diverse tasks)",
        "Balanced accuracy/speed", "Edge / autoregressive",
    ],
}
_df = pd.DataFrame(_data)

_header = "| " + " | ".join(_df.columns) + " |"
_sep = "| " + " | ".join(["---"] * len(_df.columns)) + " |"
_rows_md = "\n".join(
    "| " + " | ".join(str(v) for v in row) + " |"
    for row in _df.itertuples(index=False)
)
comparison_table = mo.md(f"""
## Method Comparison

{_header}
{_sep}
{_rows_md}
""")
comparison_table
```

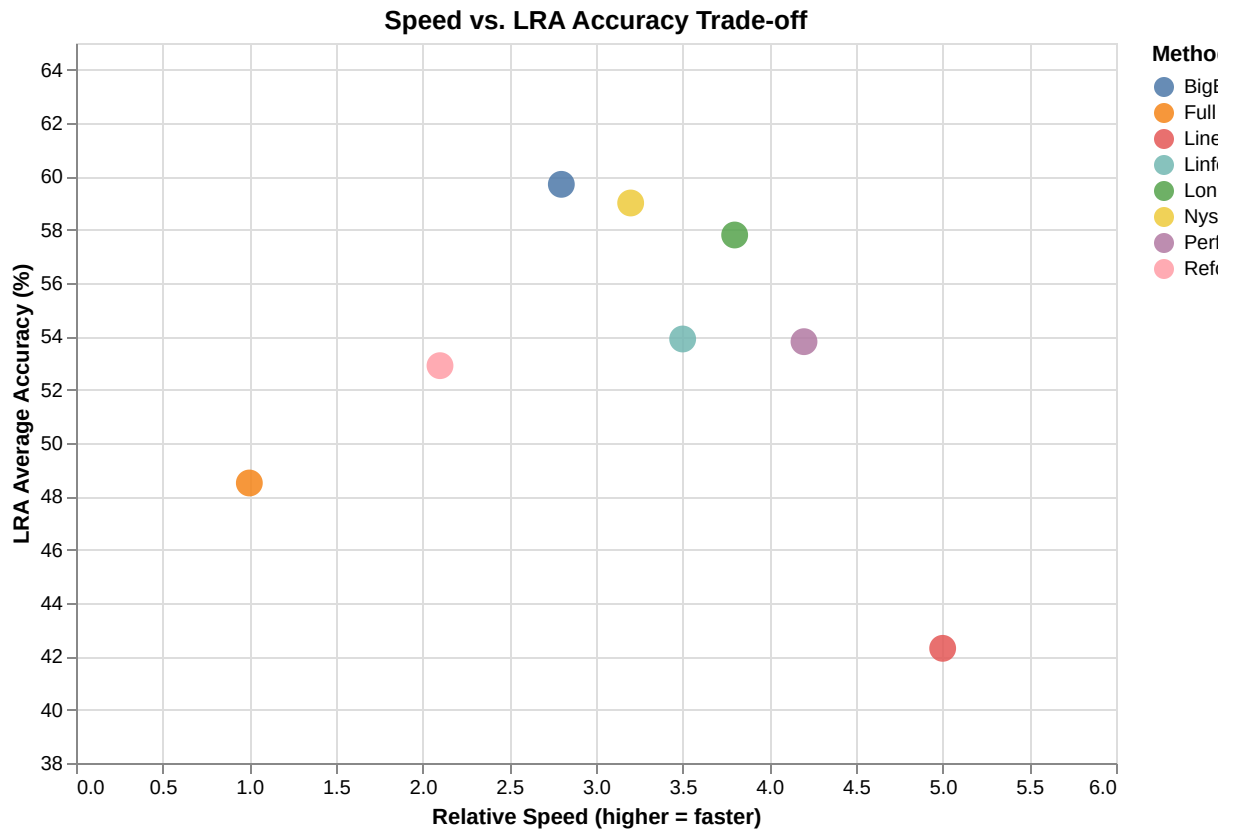
Method Comparison

Method	Complexity	Memory	Approximation	Best For
Full Transformer	$O(n^2)$	$O(n^2)$	None (exact)	Short sequences, fine-tuning
Linformer	$O(n)$	$O(nk)$	Low-rank projection	Long docs (classification)
Performer	$O(n)$	$O(nr)$	Random features	Streaming / generation
Reformer	$O(n \log n)$	$O(n \log n)$	LSH bucketing	Very long sequences
Longformer	$O(n \cdot w)$	$O(n \cdot w)$	Local + global	Document NLU
BigBird	$O(n)$	$O(n)$	Local + random + global	Long docs (diverse tasks)
Nyströmformer	$O(n)$	$O(nm)$	Nyström landmarks	Balanced accuracy/speed
Linear Transformer	$O(n)$	$O(r \cdot d)$	Kernel feature map	Edge / autoregressive

```
In [ ]: _lra_data = pd.DataFrame({
    "Method": [
        "Full Transformer", "Linformer", "Performer",
        "Reformer", "Longformer", "BigBird",
        "Nyströmformer", "Linear Transformer",
    ],
    "speed": [1.0, 3.5, 4.2, 2.1, 3.8, 2.8, 3.2, 5.0], # relative to full t
    "accuracy": [48.5, 53.9, 53.8, 52.9, 57.8, 59.7, 59.0, 42.3],
})

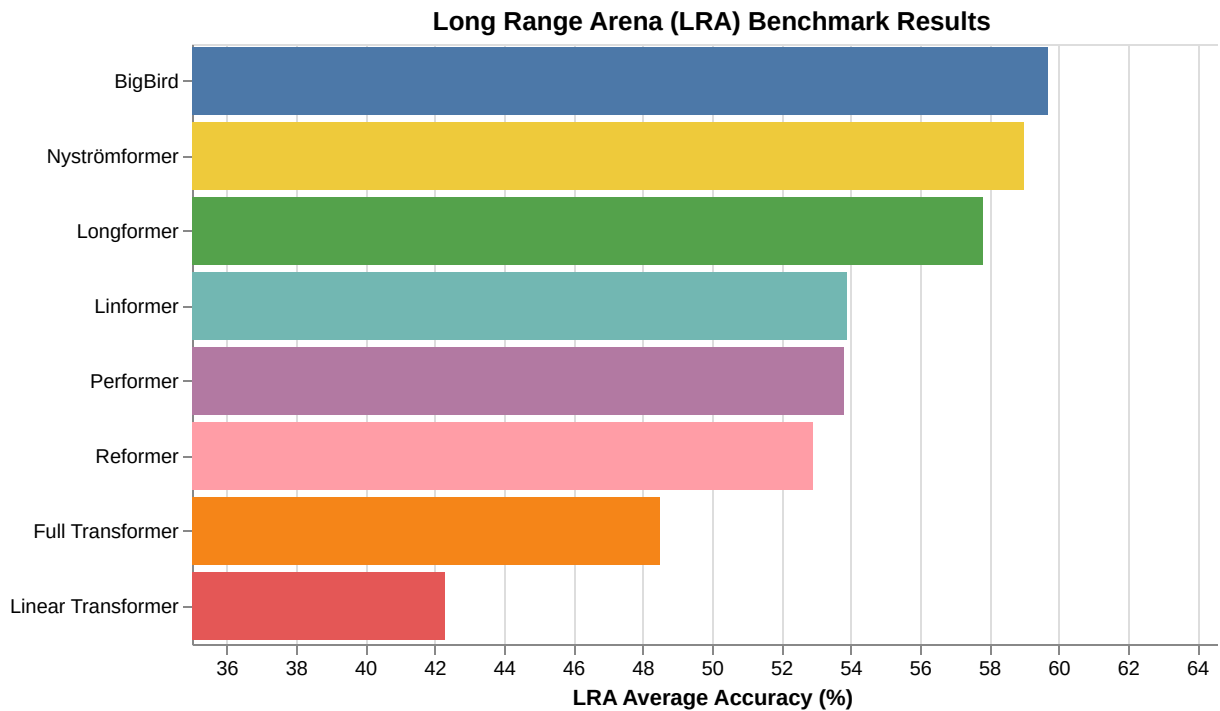
speed_accuracy_chart = (
    alt.Chart(_lra_data)
    .mark_circle(size=180, opacity=0.85)
    .encode(
        x=alt.X("speed:Q", title="Relative Speed (higher = faster)",
            scale=alt.Scale(domain=[0, 6])),
        y=alt.Y("accuracy:Q", title="LRA Average Accuracy (%)",
            scale=alt.Scale(domain=[38, 65])),
        color=alt.Color("Method:N", scale=alt.Scale(scheme="tableau10")),
        tooltip=["Method:N",
            alt.Tooltip("speed:Q", format=".1f"),
            alt.Tooltip("accuracy:Q", format=".1f")],
    )
    .properties(
        title="Speed vs. LRA Accuracy Trade-off",
        width=520,
        height=360,
    )
    .interactive()
```

```
)
speed_accuracy_chart
```



```
In [ ]: _bar_data = pd.DataFrame({
    "Method": [
        "BigBird", "Nyströmformer", "Longformer",
        "Linformer", "Performer", "Reformer",
        "Full Transformer", "Linear Transformer",
    ],
    "LRA Score": [59.7, 59.0, 57.8, 53.9, 53.8, 52.9, 48.5, 42.3],
})

lra_bar_chart = (
    alt.Chart(_bar_data)
    .mark_bar()
    .encode(
        x=alt.X("LRA Score:Q", title="LRA Average Accuracy (%)",
            scale=alt.Scale(domain=[35, 65])),
        y=alt.Y("Method:N", sort="-x", title=None),
        color=alt.Color("Method:N", scale=alt.Scale(scheme="tableau10")),
        tooltip=["Method:N", alt.Tooltip("LRA Score:Q", format=".1f")],
    )
    .properties(
        title="Long Range Arena (LRA) Benchmark Results",
        width=520,
        height=300,
    )
    .interactive()
)
lra_bar_chart
```



Implementation Notes & References

Practical Recommendations

- **Sequences < 2 K tokens:** Use standard full attention (FlashAttention for speed).
- **Sequences 2 K - 16 K, document tasks:** Longformer or BigBird are robust choices.
- **Sequences > 16 K, classification:** Nyströmformer or Linformer work well.
- **Autoregressive / streaming:** Linear Transformer or Performer with causal prefix sums.
- **Unknown task distribution:** BigBird is the safest default among sparse methods.

FlashAttention (Honorable Mention)

FlashAttention (Dao et al., 2022) is *not* an approximation — it computes exact attention in $\mathcal{O}(n^2)$ time but with $\mathcal{O}(n)$ **HBM memory** by tiling operations to stay in SRAM. It is 2-4× faster than naive attention in practice and is now the standard implementation in most frameworks.

Key Papers

Paper	Year	arXiv
Linformer	2020	2006.04768

Paper	Year	arXiv
Reformer	2020	2001.04451
Longformer	2020	2004.05150
Performer (FAVOR+)	2021	2009.14794
BigBird	2020	2007.14062
Nyströmformer	2021	2102.03902
Linear Transformer	2020	2006.16236
FlashAttention	2022	2205.14135
LRA Benchmark	2020	2011.04006

Long Range Arena Tasks

The LRA benchmark evaluates models on: **ListOps** (hierarchical reasoning), **Text** (byte-level sentiment), **Retrieval** (document similarity), **Image** (sequential CIFAR-10), **Pathfinder** (long-range spatial dependencies), and **Path-X** (extreme length version). Scores reported here are averages across all tasks from published papers.